Code Contribution and Result Details

1. Sadman Bin Anwar Siam (22301514)

Code

```python
#Sadman Bin Anwar Siam (22301514)
# ANALYSIS 4: Bivariate Interactions (Top 3 Features)
print("Generating Pairplot...")
top_features = ["V2", "V4", "V11"]

pair_plot = sns.pairplot(
    df_sample,
    vars=top_features,
    hue="Class",
    palette={0: "#3498db", 1: "#e74c3c"},
    plot_kws={"alpha": 0.5, "s": 15},
    diag_kind="kde",
)
pair_plot.fig.suptitle("Bivariate Interactions of Top Correlated Features", y=1.02)
plt.savefig("results\\dataset_feature_interaction(Bivariate_Interactions).png")
```

```python
#Sadman Bin Anwar Siam (22301514)
# Building/Trainig Simple DNN Model

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

X_train_tensor = torch.FloatTensor(X_train_scaled)
y_train_tensor = torch.FloatTensor(
    y_train.values if hasattr(y_train, "values") else y_train
).unsqueeze(1)
X_test_tensor = torch.FloatTensor(X_test_scaled)
y_test_tensor = torch.FloatTensor(
    y_test.values if hasattr(y_test, "values") else y_test
).unsqueeze(1)

val_size = int(0.2 * len(X_train_tensor))
train_size = len(X_train_tensor) - val_size
train_dataset, val_dataset = torch.utils.data.random_split(
    TensorDataset(X_train_tensor, y_train_tensor), [train_size, val_size]
)
train_loader = DataLoader(train_dataset, batch_size=128, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=128, shuffle=False)
```

```python
class DNNClassifier(nn.Module):
    def __init__(self, input_dim):
        super().__init__()
        self.network = nn.Sequential(
            nn.Linear(input_dim, 64),
            nn.ReLU(),
            nn.BatchNorm1d(64),
            nn.Dropout(0.3),
            nn.Linear(64, 32),
            nn.ReLU(),
            nn.BatchNorm1d(32),
            nn.Dropout(0.3),
            nn.Linear(32, 1),
            nn.Sigmoid(),
        )

    def forward(self, x):
        return self.network(x)


input_dim = X_train_scaled.shape[1]
model = DNNClassifier(input_dim)

criterion = nn.BCELoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)


print("Training DNN")
```

```python
best_val_loss = float("inf")
patience, patience_counter = 4, 0
best_weights = None

for epoch in range(20):
    model.train()
    for X_batch, y_batch in train_loader:
        optimizer.zero_grad()
        loss = criterion(model(X_batch), y_batch)
        loss.backward()
        optimizer.step()

    model.eval()
    val_losses = []
    with torch.no_grad():
        for X_batch, y_batch in val_loader:
            val_losses.append(criterion(model(X_batch), y_batch).item())
    val_loss = np.mean(val_losses)

    print(f"Epoch {epoch + 1:2d}/20 — val_loss: {val_loss:.4f}")

    if val_loss < best_val_loss:
        best_val_loss = val_loss
        best_weights = {k: v.clone() for k, v in model.state_dict().items()}
        patience_counter = 0
    else:
        patience_counter += 1
        if patience_counter >= patience:
            print(f"Early stopping at epoch {epoch + 1}")
            break

model.load_state_dict(best_weights)
```

```python
print("\n--- Testing the DNN on Unseen Data ---")
model.eval()
with torch.no_grad():
    dnn_probabilities = model(X_test_tensor).numpy()

dnn_predictions = (dnn_probabilities > 0.5).astype(int)

report = classification_report(y_test, dnn_predictions)
print("Deep Neural Network Results:")
print(report)
with open("results\\dnn_report.txt", "w") as f:
    f.write("Deep Neural Network Results:\n")
    f.write(report)

torch.save(
    {
        "model_state_dict": best_weights,
        "model_config": {"input_dim": input_dim},
        "scaler": scaler,
    },
    "results\\dnn_model.pt",
)
print("Model saved to dnn_model.pt")
```

```
cm = confusion_matrix(y_test, dnn_predictions)
plt.figure(figsize=(6, 4))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    cbar=False,
    xticklabels=["Legitimate", "Fraud"],
    yticklabels=["Legitimate", "Fraud"],
)
plt.title("Confusion Matrix - Deep Neural Network")
plt.ylabel("Actual Class")
plt.xlabel("Predicted Class")
plt.savefig("results\\dnn_confusion_matrix.png")

# Load DNN Model
checkpoint = torch.load("results\\dnn_model.pt", weights_only=False)

loaded_model = DNNClassifier(checkpoint["model_config"]["input_dim"])
loaded_model.load_state_dict(checkpoint["model_state_dict"])
loaded_model.eval()

scaler = checkpoint["scaler"]
print("Model loaded successfully")
```

```
#Sadman Bin Anwar Siam (22301514)
# ALGORITHMIC THRESHOLD OPTIMIZATION
print("\n--- Algorithmic Threshold Optimization ---")
precisions, recalls, thresholds = precision_recall_curve(
    y_real_world, dnn_rw_probabilities
)
f1_scores = 2 * (precisions * recalls) / (precisions + recalls + 1e-10)
# Find optimal threshold
optimal_idx = np.argmax(f1_scores)
optimal_threshold = thresholds[optimal_idx]
optimal_f1 = f1_scores[optimal_idx]
print(f"Algorithm Complete!")
print(f"Mathematically optimal threshold: {optimal_threshold:.4f}")
print(f"Expected F1-Score: {optimal_f1:.4f}")

plt.figure(figsize=(10, 6))
plt.plot(
    thresholds, f1_scores[:-1], label="F1-Score Curve", color="purple", linewidth=2
)
plt.axvline(
    x=optimal_threshold,
    color="red",
    linestyle="--",
    label=f"Optimal Threshold ({optimal_threshold:.4f})",
)
plt.title("Algorithmic Search for Optimal Decision Threshold")
plt.xlabel("Probability Threshold")
plt.ylabel("F1-Score")
plt.legend()
plt.tight_layout()
plt.savefig("results\\threshold_optimization_curve.png")
```

```python
cm_optimal = confusion_matrix(y_real_world, dnn_rw_predictions_optimal)
plt.figure(figsize=(6, 4))
sns.heatmap(
    cm_optimal,
    annot=True,
    fmt="d",
    cmap="Purples",
    cbar=False,
    xticklabels=["Legitimate", "Fraud"],
    yticklabels=["Legitimate", "Fraud"],
)
plt.title(f"DNN Confusion Matrix (Optimal Threshold: {optimal_threshold:.4f})")
plt.ylabel("Actual Class")
plt.xlabel("Predicted Class")
plt.savefig("results\\dnn_optimal_confusion_matrix.png")

print("\nAll Complete")
```

```python
#Sadman Bin Anwar Siam (22301514)
# 3. WRAPPER FUNCTION FOR LIME AND SHAP
def predict_proba_wrapper(numpy_data):
    tensor_data = torch.FloatTensor(numpy_data)
    with torch.no_grad():
        fraud_probs = model(tensor_data).numpy().flatten()

    legit_probs = 1.0 - fraud_probs
    return np.vstack((legit_probs, fraud_probs)).T


# 4. SHAP
print("\nGenerating SHAP Explanations")

background_data = shap.sample(X_scaled, 100)
test_data = shap.sample(X_scaled, 200)

explainer_shap = shap.KernelExplainer(
    lambda x: predict_proba_wrapper(x)[:, 1],
    background_data,
)

shap_values = explainer_shap.shap_values(test_data)

plt.figure(figsize=(10, 6))
shap.summary_plot(shap_values, test_data, feature_names=feature_names, show=False)
plt.title("SHAP Summary Plot: Global Feature Importance")
plt.tight_layout()
plt.savefig("results\\xai_shap_summary.png")
print("Saved SHAP plot to results\\xai_shap_summary.png")
plt.clf()
```

```python
# 5. LIME
print("\nGenerating LIME Explanation for a single Fraudulent transaction")

fraud_indices = np.where(y_sample == 1)[0]
sample_idx = fraud_indices[0]
sample_transaction_scaled = X_scaled[sample_idx]

explainer_lime = lime.lime_tabular.LimeTabularExplainer(
    training_data=X_scaled,
    feature_names=feature_names,
    class_names=["Legitimate", "Fraud"],
    mode="classification",
    random_state=42,
)

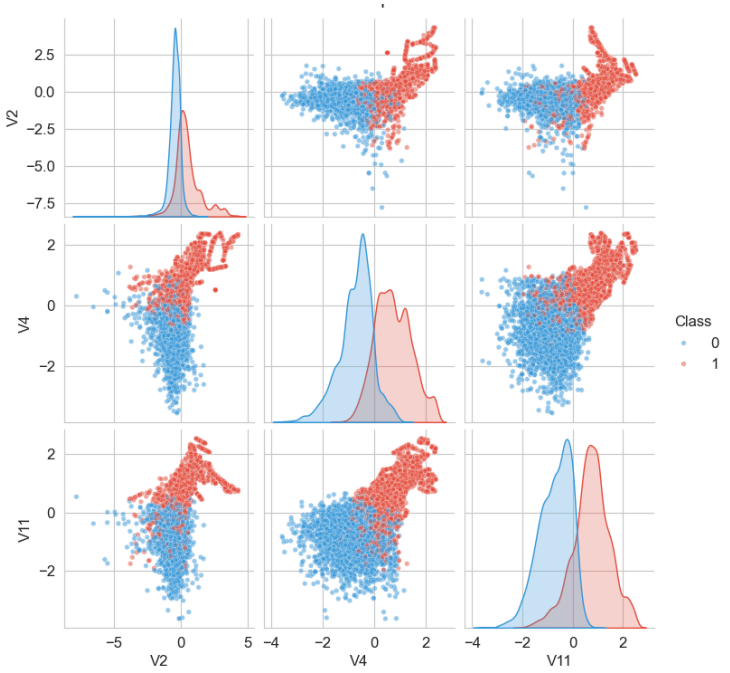
exp = explainer_lime.explain_instance(
    data_row=sample_transaction_scaled,
    predict_fn=predict_proba_wrapper,
    num_features=6,
)

fig = exp.as_pyplot_figure()
plt.title(f"LIME Explanation for Fraudulent Transaction (ID: {sample_idx})", y=1.05)
plt.tight_layout()
plt.savefig("results\\xai_lime_explanation.png")
print("Saved LIME plot to results\\xai_lime_explanation.png")

print("\nXAI Complete.")
```

Result

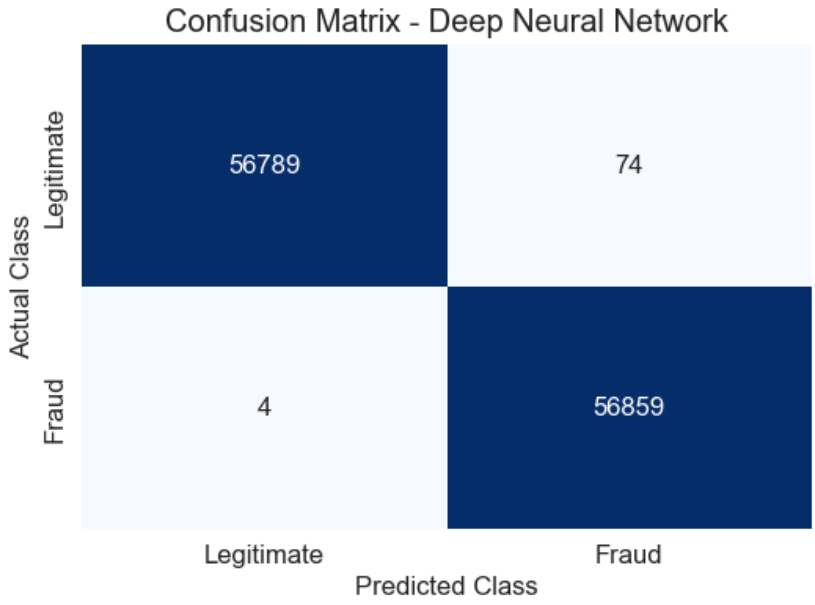
Bivariate Interactions of Top Correlated Features



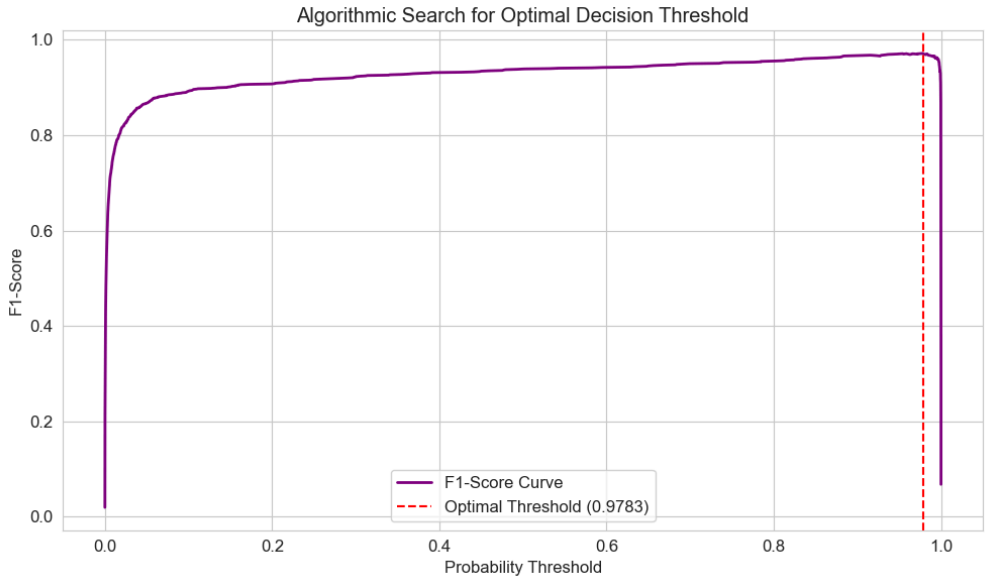
Deep Neural Network Results

Deep Neural Network Results:

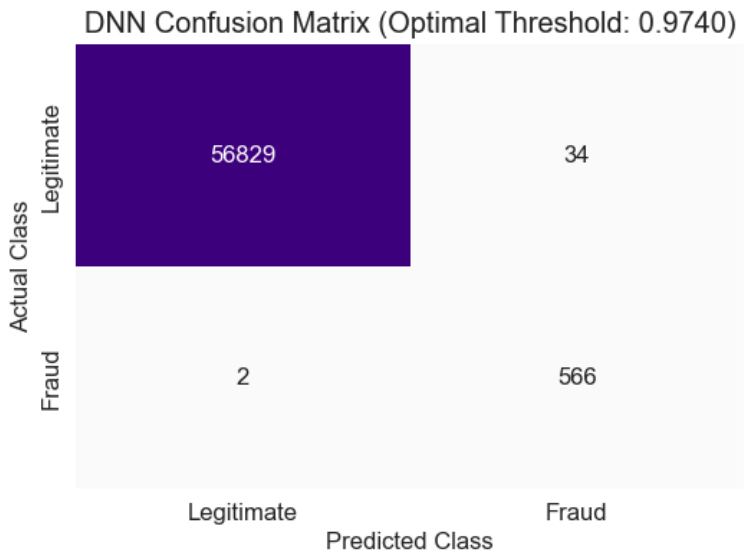
	precision	recall	f1-score	support
0	1.00	1.00	1.00	56863
1	1.00	1.00	1.00	56863
accuracy			1.00	113726
macro avg	1.00	1.00	1.00	113726
weighted avg	1.00	1.00	1.00	113726



Algorithmic Threshold Optimization to find the optimal Threshold



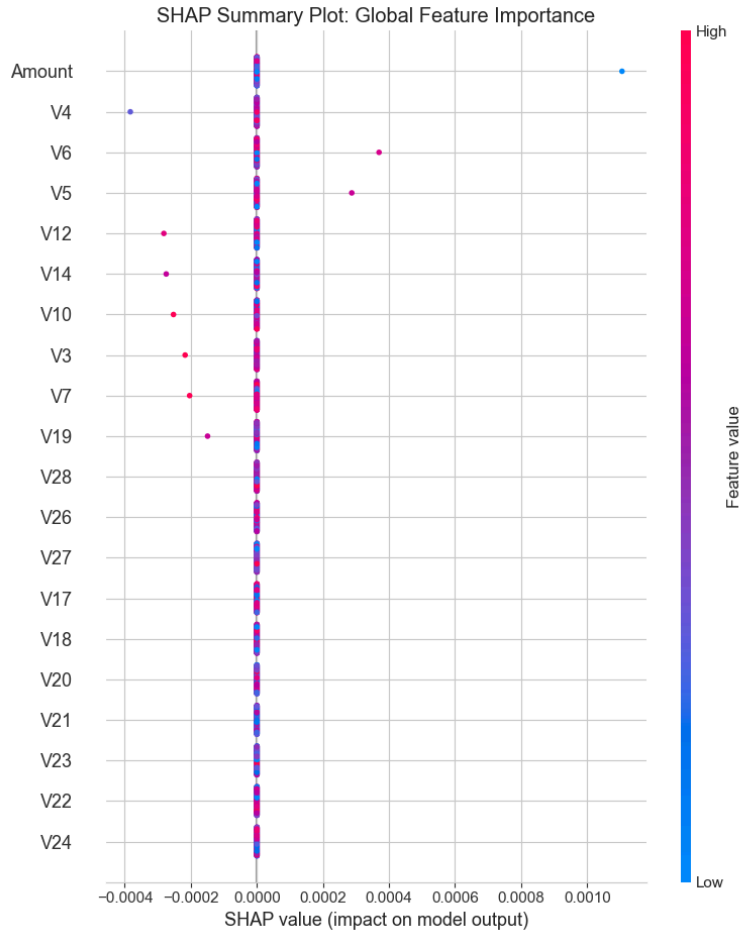
Deep Neural Network Model tested on (Real World Simulated Dataset where Fraud is 1%)



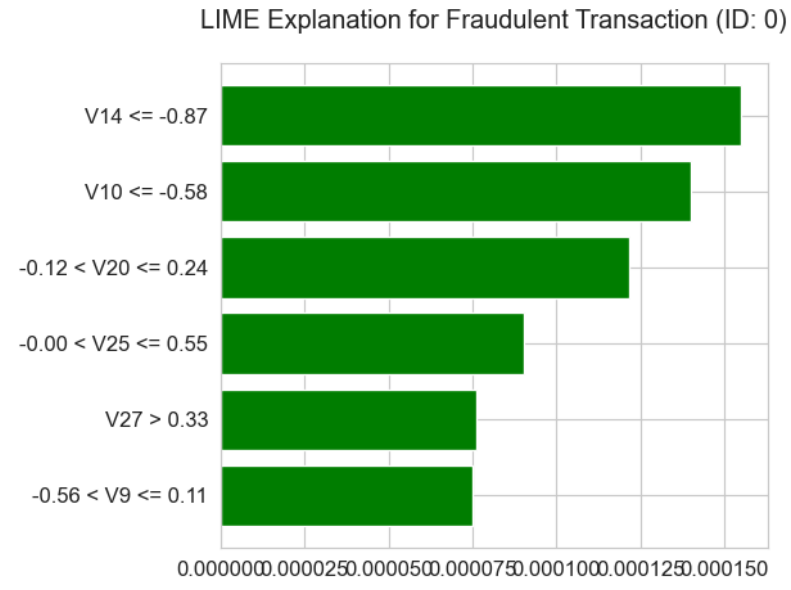
DNN Real-World Results With Optimal Threshold (Threshold 0.9740):

	precision	recall	f1-score	support
0	1.00	1.00	1.00	56863
1	0.94	1.00	0.97	568
accuracy			1.00	57431
macro avg	0.97	1.00	0.98	57431
weighted avg	1.00	1.00	1.00	57431

XAI(SHAP) Graph indicating the Important Features that drive the Decision



XAI(LIME) Graph indicating the Important Features that drive the Decision



2. A Z M Bodruddoza Bijoy (22301678)

Code

```python
#A Z M Bodruddoza Bijoy (22301678)
# 1. Load the dataset
df = pd.read_csv("dataset\\creditcard_2023.csv")

# 2. Basic Dataset Info
print("Dataset Shape:", df.shape)
print("Dataset Column Name:", df.columns)
print("\nClass Distribution:")
print(df["Class"].value_counts(normalize=True) * 100)

# 3. Drop the 'id' column
if "id" in df.columns:
    df = df.drop("id", axis=1)

# 4. Scale the 'Amount' column
scaler = RobustScaler()
df["Amount"] = scaler.fit_transform(df["Amount"].values.reshape(-1, 1))
print("\nPreprocessing complete. Data is ready for modeling.")
```

```python
#A Z M Bodruddoza Bijoy (22301678)
# ANALYSIS 1: Correlation Matrix
plt.figure(figsize=(12, 8))
corr = df.corr()
corr_with_class = corr["Class"].drop("Class").sort_values()
corr_with_class.plot(
    kind="bar", color=np.where(corr_with_class > 0, "#e74c3c", "#3498db")
)
plt.title("Correlation of Anonymized Features (V1-V28) and Amount with Fraud (Class)")
plt.ylabel("Pearson Correlation Coefficient")
plt.xlabel("Features")
plt.tight_layout()
plt.savefig("results\\dataset_feature_correlation_matrix.png")
```

```python
#A Z M Bodruddoza Bijoy (22301678)
# Split features (X) and target (y)
X = df.drop("Class", axis=1)
y = df["Class"]

# 2. Train-Test Split
# 80% of the data to train and 20% to test.
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print(f"Training data shape: {X_train.shape}")
print(f"Testing data shape: {X_test.shape}\n")
```

```python
#A Z M Bodruddoza Bijoy (22301678)
# 2. LOAD SAVED MODEL, SCALER, AND DATA
print("Loading model and scaler from dnn_model.pt...")
checkpoint = torch.load("results\\dnn_model.pt", weights_only=False)
scaler = checkpoint["scaler"]
input_dim = checkpoint["model_config"]["input_dim"]

model = DNNClassifier(input_dim)
model.load_state_dict(checkpoint["model_state_dict"])
model.eval()

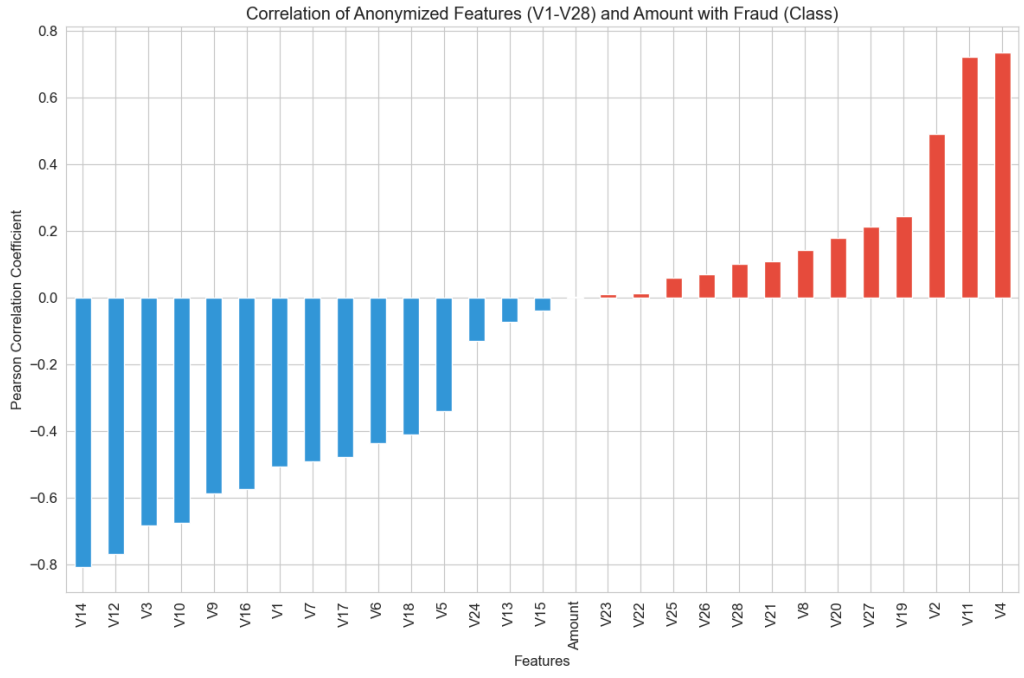
print("Loading dataset...")
df = pd.read_csv("dataset\\creditcard_2023.csv")
if "id" in df.columns:
    df = df.drop("id", axis=1)

X = df.drop("Class", axis=1)
y = df["Class"]
feature_names = X.columns.tolist()

df_sample = df.sample(n=5000, random_state=42)
X_sample = df_sample.drop("Class", axis=1)
y_sample = df_sample["Class"]

X_scaled = scaler.transform(X_sample)
X_tensor = torch.FloatTensor(X_scaled)
```

Result



3. Shabiba Binta Shahadat (22101907)

Code

```python
#Shabiba Binta Shahadat (22101907)
# ANALYSIS 2: Transaction Amount Comparison
plt.figure(figsize=(10, 6))
sns.boxplot(x="Class", y="Amount", data=df, showfliers=False, palette="Set2")
plt.title("Transaction Amount Distribution: Legitimate (0) vs Fraudulent (1)")
plt.xticks([0, 1], ["Legitimate", "Fraudulent"])
plt.ylabel("Transaction Amount")
plt.savefig("results\\dataset_transaction_boxplot.png")
```

```python
#Shabiba Binta Shahadat (22101907)
# ANALYSIS 3: Feature Distribution (Density Plots)
# take two features that had high correlation from Analysis 1

features_to_plot = ["V11", "V4"]
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
for i, feature in enumerate(features_to_plot):
    sns.kdeplot(
        data=df[df["Class"] == 0][feature],
        label="Legitimate",
        fill=True,
        ax=axes[i],
        color="blue",
    )
    sns.kdeplot(
        data=df[df["Class"] == 1][feature],
        label="Fraudulent",
        fill=True,
        ax=axes[i],
        color="red",
    )
    axes[i].set_title(f"Distribution of Feature {feature}")
    axes[i].legend()

plt.tight_layout()
plt.savefig("results\\dataset_feature_distribution.png")
```

```python
#Shabiba Binta Shahadat (22101907)
# MODEL 1: Logistic Regression
print("--- Training Logistic Regression ---")
lr_model = LogisticRegression(max_iter=1000)
lr_model.fit(X_train, y_train)
lr_predictions = lr_model.predict(X_test)

report = classification_report(y_test, lr_predictions)
print("Logistic Regression Results:")
print(report)
with open("results\\logistic_regression_report.txt", "w") as f:
    f.write("Logistic Regression Results:\n")
    f.write(report)

cm = confusion_matrix(y_test, lr_predictions)
plt.figure(figsize=(6, 4))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    cbar=False,
    xticklabels=["Legitimate", "Fraud"],
    yticklabels=["Legitimate", "Fraud"],
)
plt.title("Confusion Matrix - Logistic Regression")
plt.ylabel("Actual Class")
plt.xlabel("Predicted Class")
plt.savefig("results\\logistic_regression_confusion_matrix.png")
```

```python
#Shabiba Binta Shahadat (22101907)
# REAL-WORLD SIMULATION
print("\n--- Creating Real-World Imbalanced Test Set (99:1) ---")
test_df = X_test.copy()
test_df["Class"] = y_test

legit_test = test_df[test_df["Class"] == 0]
fraud_test = test_df[test_df["Class"] == 1]

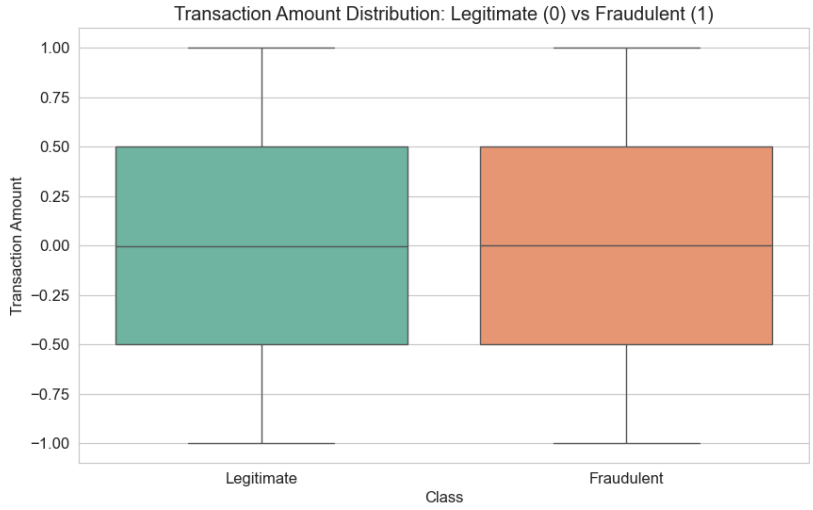
# only 1% of fraud
num_fraud_to_keep = int(len(legit_test) * 0.01)
fraud_test_sampled = fraud_test.sample(n=num_fraud_to_keep, random_state=42)

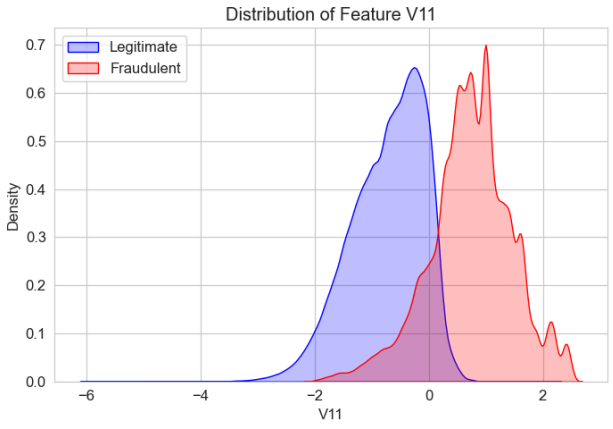
real_world_test = pd.concat([legit_test, fraud_test_sampled])
X_real_world = real_world_test.drop("Class", axis=1)
y_real_world = real_world_test["Class"]

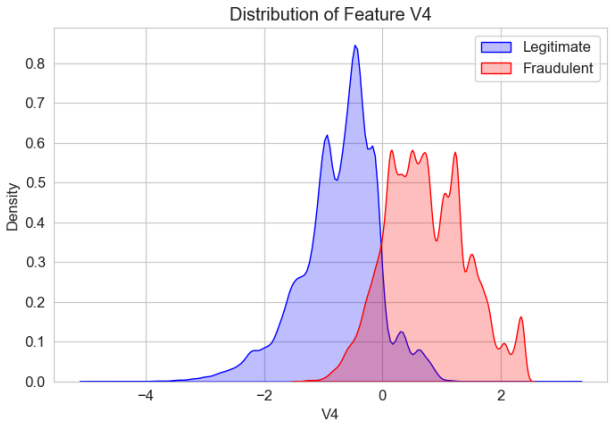
print(f"Real-World Test Set Shape: {X_real_world.shape}")
print(f"Legitimate (0): {len(legit_test)} | Fraud (1): {len(fraud_test_sampled)}")

# Process Real-World Data for PyTorch
X_rw_scaled = scaler.transform(X_real_world)
X_rw_tensor = torch.FloatTensor(X_rw_scaled)
```

Result





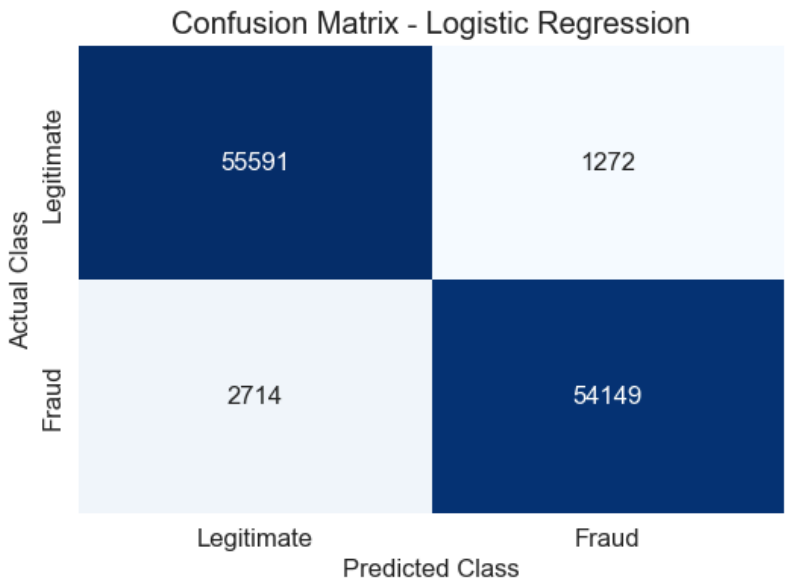


Logistic Regression Model Results

```
Logistic Regression Results:
              precision    recall  f1-score   support

           0       0.95      0.98      0.97     56863
           1       0.98      0.95      0.96     56863

    accuracy                           0.96    113726
   macro avg       0.97      0.96      0.96    113726
weighted avg       0.97      0.96      0.96    113726
```



4. Fariha Yasmin (22301331)

Code

```python
#Fariha Yasmin (22301331)
# ANALYSIS 5: t-SNE Manifold Visualization
print("Generating t-SNE Visualization")
X_sample = df_sample.drop("Class", axis=1)
y_sample = df_sample["Class"]
tsne = TSNE(n_components=2, random_state=42, perplexity=30, max_iter=1000)
X_tsne = tsne.fit_transform(X_sample)

plt.figure(figsize=(10, 8))
sns.scatterplot(
    x=X_tsne[:, 0],
    y=X_tsne[:, 1],
    hue=y_sample,
    palette={0: "#3498db", 1: "#e74c3c"},
    alpha=0.6,
    s=20,
)
plt.title("t-SNE Projection of Transaction Data (10,000 sample)")
plt.xlabel("t-SNE Dimension 1")
plt.ylabel("t-SNE Dimension 2")
plt.legend(title="Class", labels=["Legitimate (0)", "Fraudulent (1)"])
plt.savefig("results\\dataset_feature_t-SNE_visualization.png")
```

```python
#Fariha Yasmin (22301331)
# MODEL 2: Random Forest
print("\n--- Training Random Forest ---")
rf_model = RandomForestClassifier(
    n_estimators=100, max_depth=10, random_state=42, n_jobs=-1
)
rf_model.fit(X_train, y_train)
rf_predictions = rf_model.predict(X_test)

report = classification_report(y_test, rf_predictions)
print("Random Forest Results:")
print(report)
with open("results\\random_forest_report.txt", "w") as f:
    f.write("Random Forest Results:\n")
    f.write(report)

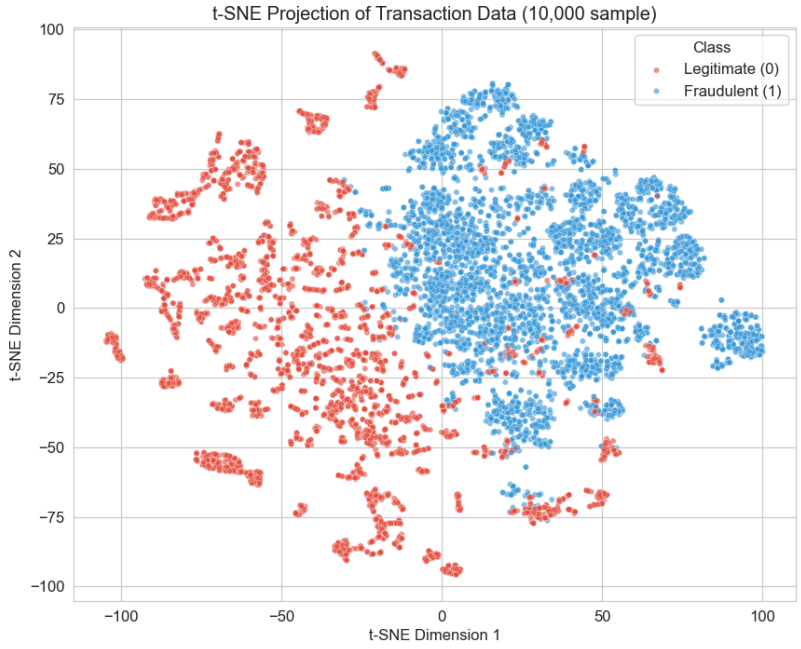
cm = confusion_matrix(y_test, rf_predictions)
plt.figure(figsize=(6, 4))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    cbar=False,
    xticklabels=["Legitimate", "Fraud"],
    yticklabels=["Legitimate", "Fraud"],
)
plt.title("Confusion Matrix - Random Forest")
plt.ylabel("Actual Class")
plt.xlabel("Predicted Class")
plt.savefig("results\\random_forest_confusion_matrix.png")
```

```python
#Fariha Yasmin (22301331)
with torch.no_grad():
    dnn_rw_probabilities = loaded_model(X_rw_tensor).numpy().flatten()

dnn_rw_predictions_default = (dnn_rw_probabilities > 0.5).astype(int)

report_rw = classification_report(y_real_world, dnn_rw_predictions_default)
print("\nDeep Neural Network Real-World Results (Threshold 0.5):")
print(report_rw)
with open("results\\dnn_real_world_report.txt", "w") as f:
    f.write("DNN Real-World Results (Threshold 0.5):\n")
    f.write(report_rw)
```

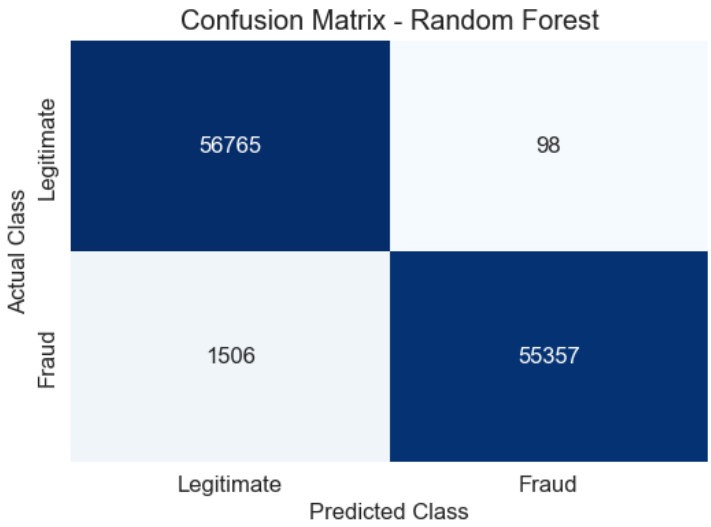
Result



Random Forest Model Results

Random Forest Results:

	precision	recall	f1-score	support
0	0.97	1.00	0.99	56863
1	1.00	0.97	0.99	56863
accuracy			0.99	113726
macro avg	0.99	0.99	0.99	113726
weighted avg	0.99	0.99	0.99	113726



Deep Neural network Result(Real World Simulated Data where Fraud is 1% with fixed Threshold 0.5)

DNN Real-World Results (Threshold 0.5):

	precision	recall	f1-score	support
0	1.00	1.00	1.00	56863
1	0.88	1.00	0.94	568
accuracy			1.00	57431
macro avg	0.94	1.00	0.97	57431
weighted avg	1.00	1.00	1.00	57431

